```
                    OBİLET OTOBÜS BİLETİ SATIŞ SİSTEMİ




                            [Ad SOYAD]
                           [Öğrenci No]



   BİLGİSAYAR TEKNOLOJİSİ VE BİLİŞİM SİSTEMLERİ BÖLÜMÜ



                              BTS304
                 Veritabanı Yönetim Sistemleri II
                           Final Ödevi



                           Haziran 2025
                              Bartın
```

ADIM-1: Senaryo

OBilet, Türkiye genelinde faaliyet gösteren bir otobüs bileti satış sistemidir. Bu sistem, kullanıcıların çevrimiçi olarak otobüs bileti araması, satın alması, iptal etmesi ve bilet geçmişini görüntülemesi gibi temel işlevleri sağlar.

Bu uygulama; - Kullanıcıların sisteme kayıt olması ve giriş yapması, - Otobüs firmalarının ve seferlerinin yönetimi, - Bilet arama ve satın alma işlemleri, - Bilet iptal ve iade işlemleri, - Kullanıcı bilet geçmişinin görüntülenmesi

gibi işlevleri yerine getirmek için tasarlanmıştır.

Uygulama Next.js framework'ü kullanılarak geliştirilmiş ve veritabanı sunucusu olarak MySQL Server kullanılmıştır. Veritabanı işlemleri için ORM kullanmak yerine, saklı yordamlar (stored procedures), kullanıcı tanımlı fonksiyonlar ve tetikleyiciler (triggers) kullanılmıştır.

Tasarladığımız veri tabanı için bazı kısıtlar aşağıda verilmiştir: - Sisteme kayıt olmayan kullanıcılar bilet satın alamaz. - Bir kullanıcı birden fazla bilet satın alabilir. - Bir sefer için belirli sayıda koltuk bulunur ve her koltuk sadece bir kez satılabilir. - Bilet satın

alındığında, ilgili seferin koltuk durumu otomatik olarak güncellenir. - Bilet iptal edildiğinde, ilgili koltuk tekrar satışa sunulur. - Kullanıcılar sadece kendi biletlerini görüntüleyebilir ve iptal edebilir. - Bilet fiyatları, mesafe, firma ve sefer saatine göre değişiklik gösterebilir. - Ödeme işlemleri sadece kredi kartı ile yapılabilir. - Hareket saatinden 3 saat öncesine kadar bilet iptali yapılabilir.

ADIM-2: Varlıklar ve Nitelikler

Kullanıcılar

- kullanici_id (PK): Kullanıcının benzersiz kimlik numarası
- tc_kimlik_no: Kullanıcının TC kimlik numarası
- ad: Kullanıcının adı
- soyad: Kullanıcının soyadı
- dogum_tarihi: Kullanıcının doğum tarihi
- cinsiyet: Kullanıcının cinsiyeti (E/K)
- telefon: Kullanıcının telefon numarası
- email: Kullanıcının e-posta adresi
- sifre: Kullanıcının şifresi (hash'lenmiş)
- kayit_tarihi: Kullanıcının sisteme kayıt olduğu tarih
- son_giris_tarihi: Kullanıcının son giriş yaptığı tarih

Firmalar

- firma_id (PK): Firmanın benzersiz kimlik numarası
- firma_adi: Firmanın adı
- logo_url: Firmanın logo URL'si
- telefon: Firmanın telefon numarası
- email: Firmanın e-posta adresi
- adres: Firmanın adresi
- website: Firmanın web sitesi
- aktif_mi: Firmanın aktif olup olmadığı (1/0)

İstasyonlar

- istasyon_id (PK): İstasyonun benzersiz kimlik numarası
- istasyon_adi: İstasyonun adı
- sehir: İstasyonun bulunduğu şehir
- adres: İstasyonun adresi
- telefon: İstasyonun telefon numarası
- aktif_mi: İstasyonun aktif olup olmadığı (1/0)

Otobüsler

- otobus_id (PK): Otobüsün benzersiz kimlik numarası
- firma_id (FK): Otobüsün ait olduğu firma
- plaka: Otobüsün plakası
- model: Otobüsün modeli
- koltuk_sayisi: Otobüsteki toplam koltuk sayısı
- wifi_var_mi: Otobüste WiFi olup olmadığı (1/0)
- tv_var_mi: Otobüste TV olup olmadığı (1/0)
- klima_var_mi: Otobüste klima olup olmadığı (1/0)
- aktif_mi: Otobüsün aktif olup olmadığı (1/0)

Seferler

- sefer_id (PK): Seferin benzersiz kimlik numarası
- firma_id (FK): Seferin ait olduğu firma
- otobus_id (FK): Seferde kullanılan otobüs
- kalkis_istasyon_id (FK): Kalkış istasyonu
- varis_istasyon_id (FK): Varış istasyonu
- kalkis_zamani: Seferin kalkış zamanı
- varis_zamani: Seferin tahmini varış zamanı
- mesafe: Seferin mesafesi (km)
- temel_ucret: Seferin temel ücreti
- aktif_mi: Seferin aktif olup olmadığı (1/0)

Koltuklar

- koltuk_id (PK): Koltuğun benzersiz kimlik numarası
- sefer_id (FK): Koltuğun ait olduğu sefer
- koltuk_no: Koltuğun numarası
- durum: Koltuğun durumu (boş/dolu)
- cinsiyet: Koltuğu satın alan kişinin cinsiyeti (E/K/null)

Biletler

- bilet_id (PK): Biletin benzersiz kimlik numarası
- sefer_id (FK): Biletin ait olduğu sefer
- kullanici_id (FK): Bileti satın alan kullanıcı
- koltuk_no: Bilet için seçilen koltuk numarası
- ucret: Bilet ücreti
- satin_alma_tarihi: Biletin satın alındığı tarih
- durum: Biletin durumu (aktif/iptal/kullanildi)

- iptal_tarihi: Biletin iptal edildiği tarih (varsa)

Ödemeler

- odeme_id (PK): Ödemenin benzersiz kimlik numarası
- bilet_id (FK): Ödemenin ait olduğu bilet
- kullanici_id (FK): Ödemeyi yapan kullanıcı
- tutar: Ödeme tutarı
- odeme_tarihi: Ödemenin yapıldığı tarih
- kart_no: Ödeme yapılan kartın son 4 hanesi
- durum: Ödemenin durumu (başarılı/başarısız/iade)

ADIM-3: Varlıklar Arası İlişkiler

Firma-Otobüs İlişkisi

- Bir firma birden fazla otobüse sahip olabilir.
- Bir otobüs yalnızca bir firmaya ait olabilir.
- İlişki: 1:N (Bir-Çok)

Firma-Sefer İlişkisi

- Bir firma birden fazla sefer düzenleyebilir.
- Bir sefer yalnızca bir firma tarafından düzenlenir.
- İlişki: 1:N (Bir-Çok)

Otobüs-Sefer İlişkisi

- Bir otobüs birden fazla seferde kullanılabilir (farklı zamanlarda).
- Bir sefer yalnızca bir otobüs tarafından gerçekleştirilir.
- İlişki: 1:N (Bir-Çok)

İstasyon-Sefer İlişkisi (Kalkış)

- Bir istasyon birden fazla seferin kalkış noktası olabilir.
- Bir seferin yalnızca bir kalkış istasyonu vardır.
- İlişki: 1:N (Bir-Çok)

İstasyon-Sefer İlişkisi (Varış)

- Bir istasyon birden fazla seferin varış noktası olabilir.
- Bir seferin yalnızca bir varış istasyonu vardır.

- İlişki: 1:N (Bir-Çok)

Sefer-Koltuk İlişkisi

- Bir sefer birden fazla koltuğa sahiptir.
- Bir koltuk yalnızca bir sefere aittir.
- İlişki: 1:N (Bir-Çok)

Kullanıcı-Bilet İlişkisi

- Bir kullanıcı birden fazla bilet satın alabilir.
- Bir bilet yalnızca bir kullanıcı tarafından satın alınabilir.
- İlişki: 1:N (Bir-Çok)

Sefer-Bilet İlişkisi

- Bir sefer için birden fazla bilet satılabilir.
- Bir bilet yalnızca bir sefere aittir.
- İlişki: 1:N (Bir-Çok)

Bilet-Ödeme İlişkisi

- Bir bilet için yalnızca bir ödeme yapılır.
- Bir ödeme yalnızca bir bilete aittir.
- İlişki: 1:1 (Bir-Bir)

Kullanıcı-Ödeme İlişkisi

- Bir kullanıcı birden fazla ödeme yapabilir.
- Bir ödeme yalnızca bir kullanıcı tarafından yapılır.
- İlişki: 1:N (Bir-Çok)

ADIM-4: ER-Şeması

ER (Entity-Relationship) şeması, veritabanı tasarımının görsel bir temsilidir. Aşağıda OBilet Otobüs Bileti Satış Sistemi için ER şeması yer almaktadır.

Not: ER şeması, https://app.diagrams.net/ kullanılarak oluşturulmuştur.

OBilet ER Şeması

ER şemasında, yukarıda tanımlanan varlıklar ve aralarındaki ilişkiler görsel olarak temsil edilmiştir. Şemada:

- Dikdörtgenler: Varlıkları (Entities) temsil eder
- Elipsler: Nitelikleri (Attributes) temsil eder
- Baklava şekilleri: İlişkileri (Relationships) temsil eder
- Çizgiler: Varlıklar ve ilişkiler arasındaki bağlantıları temsil eder

Şemada görüldüğü gibi, Kullanıcılar, Firmalar, İstasyonlar, Otobüsler, Seferler, Koltuklar, Biletler ve Ödemeler ana varlıklar olarak tanımlanmıştır. Bu varlıklar arasındaki ilişkiler, sistemin işleyişini ve veri akışını göstermektedir.

ADIM-5: Mantıksal (İlişkisel) Şema

Mantıksal şema, ER şemasının ilişkisel veritabanı modeline dönüştürülmüş halidir. Aşağıda OBilet Otobüs Bileti Satış Sistemi için mantıksal şema yer almaktadır.

- **Kullanicilar** = {kullanici_id, tc_kimlik_no, ad, soyad, dogum_tarihi, cinsiyet, telefon, email, sifre, kayit_tarihi, son_giris_tarihi}

- PK: kullanici_id

- **Firmalar** = {firma_id, firma_adi, logo_url, telefon, email, adres, website, aktif_mi}

- PK: firma_id

- **Istasyonlar** = {istasyon_id, istasyon_adi, sehir, adres, telefon, aktif_mi}

- PK: istasyon_id

- **Otobusler** = {otobus_id, firma_id, plaka, model, koltuk_sayisi, wifi_var_mi, tv_var_mi, klima_var_mi, aktif_mi}

- PK: otobus_id

- FK: firma_id -> Firmalar(firma_id)

- **Seferler** = {sefer_id, firma_id, otobus_id, kalkis_istasyon_id, varis_istasyon_id, kalkis_zamani, varis_zamani, mesafe, temel_ucret, aktif_mi}

- PK: sefer_id
- FK: firma_id -> Firmalar(firma_id)
- FK: otobus_id -> Otobusler(otobus_id)
- FK: kalkis_istasyon_id -> Istasyonlar(istasyon_id)

- FK: varis_istasyon_id -> Istasyonlar(istasyon_id)
- **Koltuklar** = {koltuk_id, sefer_id, koltuk_no, durum, cinsiyet}
- PK: koltuk_id
- FK: sefer_id -> Seferler(sefer_id)
- **Biletler** = {bilet_id, sefer_id, kullanici_id, koltuk_no, ucret, satin_alma_tarihi, durum, iptal_tarihi}
- PK: bilet_id
- FK: sefer_id -> Seferler(sefer_id)
- FK: kullanici_id -> Kullanicilar(kullanici_id)
- **Odemeler** = {odeme_id, bilet_id, kullanici_id, tutar, odeme_tarihi, kart_no, durum}
- PK: odeme_id
- FK: bilet_id -> Biletler(bilet_id)
- FK: kullanici_id -> Kullanicilar(kullanici_id)

Bu mantıksal şema, veritabanı tablolarının yapısını ve aralarındaki ilişkileri göstermektedir. Her tablonun birincil anahtarı (PK) ve yabancı anahtarları (FK) belirtilmiştir.

ADIM-6: Fiziksel Yapı

Bu bölümde, OBilet Otobüs Bileti Satış Sistemi için oluşturulan veritabanının fiziksel yapısı, SQL kodları, saklı yordamlar, kullanıcı tanımlı fonksiyonlar ve tetikleyiciler yer almaktadır.

Veritabanı Oluşturma

```
 -- Obilet Otobüs Bileti Satış Sistemi Veritabanı

-- Veritabanı oluşturma
CREATE DATABASE IF NOT EXISTS obilet_db;
USE obilet_db;

-- Kullanıcı tablosu
CREATE TABLE IF NOT EXISTS kullanici (
    kullanici_id INT AUTO_INCREMENT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    soyad VARCHAR(50) NOT NULL,
    tc_kimlik_no VARCHAR(11) NOT NULL UNIQUE,
```

```sql
    dogum_tarihi DATE NOT NULL,
    cinsiyet ENUM('E', 'K') NOT NULL,
    telefon VARCHAR(15) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    sifre VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT chk_tc_kimlik_no CHECK (LENGTH(tc_kimlik_no) =
11)
);

-- Adres bilgileri tablosu
CREATE TABLE IF NOT EXISTS adres_bilgileri (
    adres_id INT AUTO_INCREMENT PRIMARY KEY,
    kullanici_id INT NOT NULL,
    il VARCHAR(50) NOT NULL,
    ilce VARCHAR(50) NOT NULL,
    adres TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (kullanici_id) REFERENCES
kullanici(kullanici_id) ON DELETE CASCADE ON UPDATE CASCADE
);

-- Otobüs firması tablosu
CREATE TABLE IF NOT EXISTS otobus_firmasi (
    firma_id INT AUTO_INCREMENT PRIMARY KEY,
    firma_adi VARCHAR(100) NOT NULL,
    telefon VARCHAR(15) NOT NULL,
    email VARCHAR(100) NOT NULL,
    vergi_no VARCHAR(10) NOT NULL UNIQUE,
    merkez_adres TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- Otobüs tablosu
CREATE TABLE IF NOT EXISTS otobus (
    otobus_id INT AUTO_INCREMENT PRIMARY KEY,
    firma_id INT NOT NULL,
    plaka VARCHAR(10) NOT NULL UNIQUE,
    model VARCHAR(50) NOT NULL,
    koltuk_sayisi INT NOT NULL,
    ozellikler TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (firma_id) REFERENCES otobus_firmasi(firma_id)
ON DELETE CASCADE ON UPDATE CASCADE,
```

```sql
    CONSTRAINT chk_koltuk_sayisi CHECK (koltuk_sayisi > 0)
);

-- İstasyon tablosu
CREATE TABLE IF NOT EXISTS istasyon (
    istasyon_id INT AUTO_INCREMENT PRIMARY KEY,
    istasyon_adi VARCHAR(100) NOT NULL,
    il VARCHAR(50) NOT NULL,
    ilce VARCHAR(50) NOT NULL,
    adres TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- Sefer tablosu
CREATE TABLE IF NOT EXISTS sefer (
    sefer_id INT AUTO_INCREMENT PRIMARY KEY,
    otobus_id INT NOT NULL,
    kalkis_istasyon_id INT NOT NULL,
    varis_istasyon_id INT NOT NULL,
    kalkis_tarihi DATETIME NOT NULL,
    varis_tarihi DATETIME NOT NULL,
    ucret DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (otobus_id) REFERENCES otobus(otobus_id) ON
DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (kalkis_istasyon_id) REFERENCES
istasyon(istasyon_id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (varis_istasyon_id) REFERENCES
istasyon(istasyon_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_istasyon CHECK (kalkis_istasyon_id !=
varis_istasyon_id),
    CONSTRAINT chk_tarih CHECK (varis_tarihi > kalkis_tarihi),
    CONSTRAINT chk_ucret CHECK (ucret > 0)
);

-- Koltuk düzeni tablosu
CREATE TABLE IF NOT EXISTS koltuk_duzeni (
    koltuk_id INT AUTO_INCREMENT PRIMARY KEY,
    otobus_id INT NOT NULL,
    koltuk_no INT NOT NULL,
    durum ENUM('BOŞ', 'DOLU', 'REZERVE') DEFAULT 'BOŞ',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (otobus_id) REFERENCES otobus(otobus_id) ON
DELETE CASCADE ON UPDATE CASCADE,
    UNIQUE KEY uk_otobus_koltuk (otobus_id, koltuk_no),
    CONSTRAINT chk_koltuk_no CHECK (koltuk_no > 0)
```

```sql
);

-- Ödeme bilgileri tablosu
CREATE TABLE IF NOT EXISTS odeme_bilgileri (
    odeme_id INT AUTO_INCREMENT PRIMARY KEY,
    kullanici_id INT NOT NULL,
    kart_sahibi VARCHAR(100) NOT NULL,
    kart_no VARCHAR(16) NOT NULL,
    son_kullanma_tarihi VARCHAR(5) NOT NULL,
    cvv VARCHAR(3) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (kullanici_id) REFERENCES
kullanici(kullanici_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_kart_no CHECK (LENGTH(kart_no) = 16),
    CONSTRAINT chk_son_kullanma_tarihi CHECK
(LENGTH(son_kullanma_tarihi) = 5),
    CONSTRAINT chk_cvv CHECK (LENGTH(cvv) = 3)
);

-- Bilet tablosu
CREATE TABLE IF NOT EXISTS bilet (
    bilet_id INT AUTO_INCREMENT PRIMARY KEY,
    kullanici_id INT NOT NULL,
    sefer_id INT NOT NULL,
    koltuk_no INT NOT NULL,
    bilet_tarihi DATETIME DEFAULT CURRENT_TIMESTAMP,
    bilet_durumu ENUM('AKTIF', 'IPTAL', 'KULLANILDI') DEFAULT
'AKTIF',
    ucret DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (kullanici_id) REFERENCES
kullanici(kullanici_id) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (sefer_id) REFERENCES sefer(sefer_id) ON DELETE
CASCADE ON UPDATE CASCADE,
    UNIQUE KEY uk_sefer_koltuk (sefer_id, koltuk_no),
    CONSTRAINT chk_bilet_ucret CHECK (ucret > 0)
);
```

Saklı Yordamlar (Stored Procedures)

Sistemde kullanılan saklı yordamlardan bazıları aşağıda verilmiştir:

Kullanıcı İşlemleri

```
-- Kullanıcı giriş kontrolü
DELIMITER $$
CREATE PROCEDURE sp_kullanici_giris(
    IN p_email VARCHAR(100),
    IN p_sifre VARCHAR(255)
)
BEGIN
    SELECT
        kullanici_id,
        ad,
        soyad,
        tc_kimlik_no,
        dogum_tarihi,
        cinsiyet,
        telefon,
        email
    FROM kullanici
    WHERE email = p_email AND sifre = p_sifre;
END $$
DELIMITER ;

-- Kullanıcı ekle
DELIMITER $$
CREATE PROCEDURE sp_kullanici_ekle(
    IN p_ad VARCHAR(50),
    IN p_soyad VARCHAR(50),
    IN p_tc_kimlik_no VARCHAR(11),
    IN p_dogum_tarihi DATE,
    IN p_cinsiyet ENUM('E', 'K'),
    IN p_telefon VARCHAR(15),
    IN p_email VARCHAR(100),
    IN p_sifre VARCHAR(255)
)
BEGIN
    INSERT INTO kullanici (
        ad,
        soyad,
        tc_kimlik_no,
        dogum_tarihi,
        cinsiyet,
        telefon,
        email,
        sifre
    ) VALUES (
        p_ad,
        p_soyad,
        p_tc_kimlik_no,
        p_dogum_tarihi,
        p_cinsiyet,
```

```
        p_telefon,
        p_email,
        p_sifre
    );

    SELECT LAST_INSERT_ID() AS kullanici_id;
END $$
DELIMITER ;
```

Sefer İşlemleri

```
-- Sefer ara
DELIMITER $$
CREATE PROCEDURE sp_sefer_ara(
    IN p_kalkis_il VARCHAR(50),
    IN p_varis_il VARCHAR(50),
    IN p_tarih DATE
)
BEGIN
    SELECT
        s.sefer_id,
        s.otobus_id,
        o.plaka,
        f.firma_adi,
        s.kalkis_istasyon_id,
        ki.istasyon_adi AS kalkis_istasyon_adi,
        ki.il AS kalkis_il,
        s.varis_istasyon_id,
        vi.istasyon_adi AS varis_istasyon_adi,
        vi.il AS varis_il,
        s.kalkis_tarihi,
        s.varis_tarihi,
        s.ucret
    FROM sefer s
    JOIN otobus o ON s.otobus_id = o.otobus_id
    JOIN otobus_firmasi f ON o.firma_id = f.firma_id
    JOIN istasyon ki ON s.kalkis_istasyon_id = ki.istasyon_id
    JOIN istasyon vi ON s.varis_istasyon_id = vi.istasyon_id
    WHERE ki.il = p_kalkis_il
    AND vi.il = p_varis_il
    AND DATE(s.kalkis_tarihi) = p_tarih;
END $$
DELIMITER ;
```

Bilet İşlemleri

```
-- Bilet satın al
DELIMITER $$
CREATE PROCEDURE sp_bilet_satin_al(
```

```
    IN p_kullanici_id INT,
    IN p_sefer_id INT,
    IN p_koltuk_no INT,
    IN p_ucret DECIMAL(10, 2)
)
BEGIN
    DECLARE koltuk_durumu VARCHAR(10);
    DECLARE sefer_tarihi DATETIME;

    -- Seferin tarihini kontrol et
    SELECT kalkis_tarihi INTO sefer_tarihi
    FROM sefer
    WHERE sefer_id = p_sefer_id;

    IF sefer_tarihi < NOW() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Bu sefer için bilet satın alınamaz.
Sefer tarihi geçmiş.';
    END IF;

    -- Koltuğun durumunu kontrol et
    SELECT
        CASE
            WHEN b.bilet_id IS NOT NULL THEN 'DOLU'
            ELSE kd.durum
        END INTO koltuk_durumu
    FROM koltuk_duzeni kd
    JOIN sefer s ON kd.otobus_id = s.otobus_id
    LEFT JOIN bilet b ON s.sefer_id = b.sefer_id AND
kd.koltuk_no = b.koltuk_no
    WHERE s.sefer_id = p_sefer_id AND kd.koltuk_no =
p_koltuk_no;

    IF koltuk_durumu = 'DOLU' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Bu koltuk zaten satılmış.';
    END IF;

    -- Bileti oluştur
    INSERT INTO bilet (
        kullanici_id,
        sefer_id,
        koltuk_no,
        ucret
    ) VALUES (
        p_kullanici_id,
        p_sefer_id,
        p_koltuk_no,
        p_ucret
    );

    SELECT LAST_INSERT_ID() AS bilet_id;
```

```
END $$
DELIMITER ;

-- Bilet iptal et
DELIMITER $$
CREATE PROCEDURE sp_bilet_iptal(
    IN p_bilet_id INT
)
BEGIN
    DECLARE bilet_durumu VARCHAR(10);
    DECLARE sefer_tarihi DATETIME;
    DECLARE iptal_suresi INT;

    -- Biletin durumunu kontrol et
    SELECT b.bilet_durumu, s.kalkis_tarihi
    INTO bilet_durumu, sefer_tarihi
    FROM bilet b
    JOIN sefer s ON b.sefer_id = s.sefer_id
    WHERE b.bilet_id = p_bilet_id;

    IF bilet_durumu != 'AKTIF' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Bu bilet zaten iptal edilmiş veya
kullanılmış.';
    END IF;

    -- İptal süresini kontrol et (3 saat öncesine kadar iptal
edilebilir)
    SET iptal_suresi = TIMESTAMPDIFF(HOUR, NOW(), sefer_tarihi);

    IF iptal_suresi < 3 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Bilet iptal süresi geçmiş. Hareket
saatine 3 saatten az kaldığı için bilet iptal edilemez.';
    END IF;

    -- Bileti iptal et
    UPDATE bilet
    SET
        bilet_durumu = 'IPTAL',
        updated_at = NOW()
    WHERE bilet_id = p_bilet_id;
END $$
DELIMITER ;
```

Kullanıcı Tanımlı Fonksiyonlar

```
-- Seferin doluluk oranını hesaplayan fonksiyon
DELIMITER $$
CREATE FUNCTION fn_sefer_doluluk_orani(p_sefer_id INT)
```

```sql
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE toplam_koltuk INT;
    DECLARE dolu_koltuk INT;
    DECLARE doluluk_orani DECIMAL(5,2);
    DECLARE otobus_id_val INT;

    -- Seferin otobüs ID'sini al
    SELECT otobus_id INTO otobus_id_val
    FROM sefer
    WHERE sefer_id = p_sefer_id;

    -- Otobüsün toplam koltuk sayısını al
    SELECT koltuk_sayisi INTO toplam_koltuk
    FROM otobus
    WHERE otobus_id = otobus_id_val;

    -- Seferdeki dolu koltuk sayısını hesapla
    SELECT COUNT(*) INTO dolu_koltuk
    FROM bilet
    WHERE sefer_id = p_sefer_id AND bilet_durumu = 'AKTIF';

    -- Doluluk oranını hesapla
    IF toplam_koltuk > 0 THEN
        SET doluluk_orani = (dolu_koltuk / toplam_koltuk) * 100;
    ELSE
        SET doluluk_orani = 0;
    END IF;

    RETURN doluluk_orani;
END $$
DELIMITER ;

-- Bilet fiyatını hesaplayan fonksiyon (mesafe, firma ve koltuk
tipine göre)
DELIMITER $$
CREATE FUNCTION fn_bilet_fiyati_hesapla(
    p_kalkis_il VARCHAR(50),
    p_varis_il VARCHAR(50),
    p_firma_id INT,
    p_koltuk_no INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE mesafe INT;
    DECLARE baz_fiyat DECIMAL(10,2);
    DECLARE firma_katsayi DECIMAL(5,2);
    DECLARE koltuk_katsayi DECIMAL(5,2);
    DECLARE final_fiyat DECIMAL(10,2);
```

```
    -- Mesafeyi hesapla
    SET mesafe = fn_sehirler_arasi_mesafe(p_kalkis_il,
p_varis_il);

    -- Baz fiyatı hesapla (km başına 0.5 TL)
    SET baz_fiyat = mesafe * 0.5;

    -- Firma katsayısını belirle
    SELECT
        CASE
            WHEN firma_id = 1 THEN 1.2 -- Premium firma
            WHEN firma_id = 2 THEN 1.1 -- Standart firma
            ELSE 1.0 -- Diğer firmalar
        END INTO firma_katsayi
    FROM otobus_firmasi
    WHERE firma_id = p_firma_id;

    -- Koltuk katsayısını belirle (koridor veya cam kenarı)
    IF p_koltuk_no % 2 = 0 THEN
        SET koltuk_katsayi = 1.1; -- Cam kenarı
    ELSE
        SET koltuk_katsayi = 1.0; -- Koridor
    END IF;

    -- Final fiyatı hesapla
    SET final_fiyat = baz_fiyat * firma_katsayi *
koltuk_katsayi;

    RETURN ROUND(final_fiyat, 2);
END $$
DELIMITER ;
```

Tetikleyiciler (Triggers)

```
 -- Bilet eklendiğinde koltuk durumunu güncelleyen tetikleyici
DELIMITER $$
CREATE TRIGGER trg_bilet_after_insert
AFTER INSERT ON bilet
FOR EACH ROW
BEGIN
    DECLARE otobus_id_val INT;

    -- Seferin otobüs ID'sini al
    SELECT otobus_id INTO otobus_id_val
    FROM sefer
    WHERE sefer_id = NEW.sefer_id;

    -- Koltuk durumunu güncelle
    UPDATE koltuk_duzeni
    SET durum = 'DOLU'
```

```
    WHERE otobus_id = otobus_id_val AND koltuk_no =
NEW.koltuk_no;
END $$
DELIMITER ;

-- Bilet iptal edildiğinde koltuk durumunu güncelleyen
tetikleyici
DELIMITER $$
CREATE TRIGGER trg_bilet_after_update
AFTER UPDATE ON bilet
FOR EACH ROW
BEGIN
    DECLARE otobus_id_val INT;

    -- Eğer bilet durumu IPTAL olarak değiştiyse
    IF NEW.bilet_durumu = 'IPTAL' AND OLD.bilet_durumu !=
'IPTAL' THEN
        -- Seferin otobüs ID'sini al
        SELECT otobus_id INTO otobus_id_val
        FROM sefer
        WHERE sefer_id = NEW.sefer_id;

        -- Koltuk durumunu güncelle
        UPDATE koltuk_duzeni
        SET durum = 'BOŞ'
        WHERE otobus_id = otobus_id_val AND koltuk_no =
NEW.koltuk_no;
    END IF;
END $$
DELIMITER ;
```

ADIM-7: Arayüz Tasarımı ve Uygulama Geliştirme

Bu bölümde, OBilet Otobüs Bileti Satış Sistemi için geliştirilen web uygulamasının arayüz tasarımı ve uygulama geliştirme süreci anlatılmaktadır.

Kullanılan Teknolojiler

Uygulama aşağıdaki teknolojiler kullanılarak geliştirilmiştir:

- **Frontend**: Next.js, React, Tailwind CSS
- **Backend**: Next.js API Routes
- **Veritabanı**: MySQL
- **ORM**: Raw SQL (Stored Procedures, Functions, Triggers)

Uygulama Mimarisi

Uygulama, Next.js framework'ü kullanılarak geliştirilmiştir. Next.js, React tabanlı bir web framework'üdür ve hem istemci tarafı (client-side) hem de sunucu tarafı (server-side) rendering özelliklerine sahiptir. Uygulama mimarisi aşağıdaki bileşenlerden oluşmaktadır:

1. **Frontend**: React bileşenleri, sayfalar ve UI bileşenleri
2. **Backend**: Next.js API Routes ile oluşturulan API'ler
3. **Veritabanı Bağlantısı**: MySQL veritabanına bağlantı için mysql2 kütüphanesi
4. **Veritabanı İşlemleri**: Saklı yordamlar, fonksiyonlar ve tetikleyiciler

Uygulama Ekran Görüntüleri

Ana Sayfa

Ana sayfa, kullanıcıların bilet araması yapabileceği bir arama formu, popüler rotalar, neden OBilet'i tercih etmeleri gerektiğini anlatan özellikler ve müşteri yorumları içermektedir.

Ana Sayfa

Bilet Arama Formu

Bilet arama formu, kullanıcıların kalkış ve varış noktalarını seçerek, tarih belirleyerek bilet araması yapabilmelerini sağlar.

Bilet Arama Formu

Seferler Sayfası

Seferler sayfası, kullanıcının arama kriterlerine uygun seferleri listeler. Kullanıcılar, seferleri fiyat, kalkış saati veya firma adına göre filtreleyebilir ve sıralayabilir.

Seferler Sayfası

Koltuk Seçimi Sayfası

Koltuk seçimi sayfası, kullanıcının seçtiği sefer için koltuk seçimi yapabilmesini sağlar. Dolu, boş ve seçili koltuklar farklı renklerle gösterilir.

Koltuk Seçimi Sayfası

Ödeme Sayfası

Ödeme sayfası, kullanıcının seçtiği koltuk için ödeme yapabilmesini sağlar. Kullanıcı, kredi kartı bilgilerini girerek ödeme yapabilir.

Ödeme Sayfası

Kullanıcı Profil Sayfası

Kullanıcı profil sayfası, kullanıcının kişisel bilgilerini ve satın aldığı biletleri görüntüleyebilmesini sağlar. Kullanıcı, biletlerini iptal edebilir veya bilet detaylarını görüntüleyebilir.

Kullanıcı Profil Sayfası

Uygulama Özellikleri

Kullanıcı Yönetimi

- Kullanıcı kaydı ve girişi
- Kullanıcı profil bilgilerinin güncellenmesi
- Şifre değiştirme

Bilet İşlemleri

- Bilet arama (kalkış-varış noktası ve tarih seçerek)
- Sefer listeleme ve filtreleme
- Koltuk seçimi
- Bilet satın alma
- Bilet iptal etme
- Bilet geçmişi görüntüleme

Ödeme İşlemleri

- Kredi kartı ile ödeme
- Ödeme geçmişi görüntüleme

Veritabanı Entegrasyonu

Uygulama, MySQL veritabanına bağlanarak verileri saklar ve işler. Veritabanı işlemleri için saklı yordamlar, fonksiyonlar ve tetikleyiciler kullanılmıştır. Bu sayede, veritabanı işlemleri daha güvenli ve performanslı hale getirilmiştir.

Örnek bir API rotası:

```
// src/app/api/seferler/ara/route.ts
import { executeStoredProcedure } from '@/lib/db';
import { NextRequest, NextResponse } from 'next/server';

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const kalkis_il = searchParams.get('kalkis_il');
    const varis_il = searchParams.get('varis_il');
    const tarih = searchParams.get('tarih');

    if (!kalkis_il || !varis_il || !tarih) {
      return NextResponse.json(
        { error: 'Kalkış ili, varış ili ve tarih parametreleri
gereklidir.' },
        { status: 400 }
      );
    }

    const results = await
executeStoredProcedure('sp_sefer_ara', [
      kalkis_il,
      varis_il,
      tarih,
    ]);

    return NextResponse.json(results);
  } catch (error: any) {
    console.error('Sefer arama hatası:', error);
    return NextResponse.json(
      { error: 'Seferler yüklenirken bir hata oluştu.' },
      { status: 500 }
    );
  }
}
```

Sonuç

OBilet Otobüs Bileti Satış Sistemi, kullanıcıların kolayca otobüs bileti araması, satın alması, iptal etmesi ve bilet geçmişini görüntülemesi için gerekli tüm özellikleri sağlayan kapsamlı bir web uygulamasıdır. Uygulama, modern web teknolojileri kullanılarak geliştirilmiş ve kullanıcı dostu bir arayüze sahiptir.

Veritabanı tasarımı, saklı yordamlar, fonksiyonlar ve tetikleyiciler kullanılarak, veritabanı işlemleri daha güvenli ve performanslı hale getirilmiştir. Bu sayede, uygulama büyük veri setleriyle bile hızlı ve güvenilir bir şekilde çalışabilir.